

LYRA-CPP — THETIS DIRECT-PORT PLAN (TX)

Authoritative doc for the current TX work direction.

■ *The older EXECUTION_PLAN.md (Step 14 wire-layer rebuild, Phase 3 etc.) describes a DIFFERENT approach that is now superseded for the TX work. Do NOT mix the two plans — that doc tracks an earlier wire-rebuild pass which was partial and Lyra-native-leaning. THIS doc is the active plan going forward. Old doc remains in tree for historical reference only (some shipped pieces from it are still in production — noted per-component in the registry below).*

Branch: tx-rebuild (the direct-port work lives here; origin/main carries the earlier Step 14 / TX-1 line and is deliberately untouched by the direct-port arc).

Reference tree (the ONLY trusted TX reference):

- D:\sdrprojects\OpenHPSDR-Thetis-2.10.3.13\Project Files\Source\
 - wdsp*.c — WDSP DSP engine (GPL v3+, port-with-attribution OK)
 - ChannelMaster*.c — wire layer (GPL v3+, port-with-attribution OK)
 - Console*.cs — C# UI/control glue (study ONLY; write Lyra-native equivalents)

- Y:\Claude local_hl2src\ — HL2+ ak4951v4 gateway RTL

(control.v / radio.v / usopenhpsdr1.v etc. — the ground truth when host-side reference and gateway disagree)

Provenance & attribution rule (operator-locked 2026-06-09, supersedes the 2026-05-31 "no-attribution-in-commits" tightening):

Lyra-cpp is a derivative work of openHPSDR Thetis (ChannelMaster + WDSP modules), GPL v3+. The GitHub project description openly states this: **"TX baseline ported from openHPSDR Thetis (ChannelMaster) with Lyra-native DSP additions."** Open attribution is both legally required (GPL) and operator-intended (public project posture).

1. **License attribution at file head — REQUIRED.** Any file ported from WDSP or ChannelMaster carries a header block crediting upstream (NR0V for WDSP modules; openHPSDR Thetis for ChannelMaster modules) + the GPL v3+ notice. Existing AAMix.{h, cpp}, CMaster.cpp, RadioNet.cpp etc. already follow this; keep it.
2. **Code comments citing source — ENCOURAGED.** Citations may name the reference openly (Reference: Thetis ChannelMaster cmaster.c:297-313, // Per openHPSDR networkproto1.c:1078, // HL2+ ak4951v4 gateway control.v:209-220). The file:line is the primary technical content; the reference-app name is context that helps future readers find the source tree.
3. **Commit messages — REFERENCE-APP NAMES PERMITTED.** A commit like **"port aamix.c from Thetis ChannelMaster with Lyra-native idiom translations"** is fine — it matches the public GitHub posture and makes the history grep-able. The earlier 4cef88d tightening (2026-05-31) that put commit messages on the no-attribution list is **rescinded**.
4. ****Public docs (README, docs/*.md design docs, memory files, this plan) — OPEN ATTRIBUTION.**** Same posture as the GitHub description.
5. **User-facing UI strings — KEEP OPERATOR-FOCUSED.** UI labels operators see in normal operation (panel titles, button text, error toasts) describe Lyra behavior, not provenance. **"AAMix output (HL2 jack)"** not **"Thetis-porting AAMix output"**. About dialog / credits screen / install docs MAY (and probably should) name openHPSDR Thetis + WDSP/NR0V openly — credit where due. This is the only category that stays "Lyra-native voice in operator-facing labels," because operators interact with Lyra, not its source provenance.
6. **The "Lyra-Native style governs surrounding architecture only" rule (Stage B-locked) STANDS unchanged.** Reference DSP API call patterns are byte-for-byte faithful; Qt/Vulkan/QML/process model is Lyra-native.
7. **Reference precedence STANDS unchanged.** Verify TX ONLY against Thetis 2.10.3.13 (D:\sdrprojects\OpenHPSDR-Thetis-2.10.3.13\...) + the HL2+ ak4951v4 gateway RTL (Y:\Claude local_hl2src\). **Old**

Python Lyra is NOT a trusted TX reference (its TX never worked).

Consequence of the 2026-06-09 amendment: the three historical commits (efdde02, 604d87b, ed9f0cd) that "predated the no-attribution rule" and contained "Thetis" in their messages are **no longer in violation** — they're now consistent with the rule. Task #73 ("Cleanup pre-existing 'Thetis' mentions in shipped code") is **OBSOLETE**d by this amendment.

■ LOCKED METHODOLOGY (Stage B-validated, do NOT relax)

These rules were earned through the Stage B aamix.c port arc (2026-06-08) — see SESSION_LOG.md for the full trail.

1. ****Smallest revertable empirical step → operator HL2 bench →**

next step.** Each commit is one focused change with a falsifiable bench gate. Multi-step "design then implement" is for stages that genuinely need it (operator approves); default is iterate-and-bench.

2. **"Do as Thetis does, Lyra-Native style"** (locked

2026-05-30 after the fexchange0 crash arc, re-validated tonight after the B.6.b shortcut attempt cost a revert cycle). Three sub-rules:

- Every WDSP API call matches the reference byte-for-byte

(parameter values, order, the SET of setters fired, the lifecycle stage at which they fire). If tempted to pick a value the reference doesn't use, STOP and find what the reference uses. "I have a reason to pick differently" means I don't. Reference has been right every time.

- "Lyra-Native style" governs surrounding architecture ONLY

(Qt + Vulkan + QML + process/thread model + facade APIs + IPC + UI shell). The WDSP DSP-engine call surface is reference-bound.

- Provenance only in code COMMENTS + design docs + memory.

Never commit messages, never UI strings.

3. **Counters first when the symptom is "nothing happens".**

The Stage B 2-stage instrumentation (chain counters in xMixAudio/mix_main + dispatch counters in dispatchAudioFrame) localized the silent-audio defect to a single line of code on the 2nd bench. Pure speculation would have cost a multi-round revert/speculate cycle. Atomic counter + rate-limited qInfo = trivial overhead, very high diagnostic ROI when "I press play, hear nothing, log looks normal."

4. **Operator-empirical rule overrides agent inference.**

When operator hardware data contradicts what an audit/red-team/Plan-agent concluded, the data wins. Do not defend the audit — re-open, bisect, fix. This rule has fired repeatedly through the broader project arc; the Stage B B.6.b-fix1 fix was an instance (chain counters healthy suggested AAMix was correct; the temptation was to revert the whole port; the empirical pattern instead pointed at the wire-up surface where the bug actually was).

5. ****Read the reference FIRST, in shipped-code form, before**

designing.** Every time we designed first and read second, we shipped a regression. Every time we read first, we got the right answer in one pass. The no-attribution rule applies to shipped code; it does NOT mean "don't read the reference." Read it, dossier the file:line in the plan doc, then write Lyra-cpp.

6. **Session-start discipline:** `git fetch origin` is the

FIRST action every session, before reading any local state. If divergence exists, surface it to the operator BEFORE writing code. No exceptions.

7. **Push after every component ships, not at EOD.** Each

clean operator HL2 bench → backup bundle + push immediately. Smaller in-flight delta = smaller surprise blast radius.

8. **Backup bundle naming:** `_backups/lyra-cpp-YYYY-MM-DD-.bundle`.

`git bundle create ... --all` (covers all refs).

■ STATUS REGISTRY — what's done, in-progress, pending

Format: per-file or per-component row. Status emoji:

- ■ DONE + operator-bench-validated
- ■ IN-PROGRESS (committed but not yet bench-validated, or partially shipped)
- ■ PENDING (planned, not started)
- ■ N/A (Lyra-native equivalent already shipped; no direct port planned — see notes column)

Wire layer (*ChannelMaster* — port directly w/ attribution)

Source	Target (lyra-cpp)	Status	Notes
ChannelMaster\cmaster.c (stubs only)	src/wire/CMaster.{h,cpp} (Stage A)	■	Stage A stubs shipped earlier. SendpOutboundRx wired to AAMix Stage B.5. SendpOutboundTx pending Stage C ilv.c port. SendpOutboundTCIRxIQ + SendpInboundTCITxAudio stubs only.
ChannelMaster\cmaster.c::create_xmtr	src/wdsp/TxChannel.{h,cpp}	■	TxChannel class shipped on main (TX-1 components 2-6); needs verify-against-reference pass to confirm parameter parity (the 2026-05-30 fexchange0 arc forced an in-line reconciliation but TxChannel hasn't been formally registered as a direct-port).
ChannelMaster\ilv.c (TX I/Q interleaver)	src/wdsp/Ilv.{h,cpp} (NEW, Stage C candidate)	■	Pairs with the SendpOutboundTx stub already wired in CMaster Stage A. Required for the TX I/Q path to reach the EP2 packer reference-faithfully.
ChannelMaster\cmaster.c::xcmaster (pump body)	src/wire/CMaster.cpp::xcmaster (NEW, Stage D candidate)	■	Currently Lyra-cpp uses its own pump driver. Direct port would replace the Lyra-native driver loop.
ChannelMaster\networkproto1.c (HL2 wire writer)	src/hl2_stream.cpp (LIVE) + src/wire/* (Step 14 pieces)	■	This is the file the Step 14 plan was attacking. Currently a MIX of Step 14 Lyra-native rewrites + the original hl2_stream.cpp body. Direct-port verdict pending operator decision (see "Open questions" below).

Source	Target (lyra-cpp)	Status	Notes
ChannelMaster\network.c (UDP + keepalive)	src/wire/Ep2SendThread.{h,cpp} + src/wire/Ep6RecvThread.{h,cpp}	■	Step 14 Lyra-native rewrites shipped (\$5/\$6); could be re-ported direct from reference, OR left as-is if reference-parity audit passes. Operator call.
ChannelMaster\netInterface.c::create_rnet	src/wire/RadioNet.cpp::create_rnet (LIVE)	■	Direct port shipped earlier. The Stage-A SendpOutboundRx no-op stub at line 272 was DELETED 2026-06-08 (B.6.b-fix1, commit 8b8e0da) because it clobbered the AAMix wire-up; per the comment-block left in the deleted-case body, future codec-id switching for ASIO/WASAPI must NOT re-introduce a default registration here — it would re-clobber aaMix_->Outbound.

WDSP RX chain (port directly w/ attribution)

Source	Target (lyra-cpp)	Status	Notes
wdsp\aaMix.{c,h}	src/wdsp/AAMix.{h,cpp}	■	STAGE B COMPLETE 2026-06-08. Full direct port w/ NR0V/WDSP GPL v3+ attribution. 538-line header + 1100-line cpp. Bench-validated end-to-end through both AK4951 and PC-Soundcard sinks. Idiom translations documented (Win32 → C++23). Reference-faithful create_aaMix(active=0) + SetAAudioMixState(activate) pattern per cmaster.c:297-313.
wdsp\firmin.c (filter cores)	(consumed via WDSP cffi)	■	Used internally by WDSP RXA chain via cdef + dlsym. No source-level port needed (the DLL ships with Lyra-cpp via _native/ per the Python-project pattern; cdef + dlsym in wdsp_native.cpp). WISDOM cache fix shipped on Python side; lyra-cpp's RX chain inherits the same path.
wdsp\wisdom.c	(consumed via WDSP cffi)	■	Same as firmin.c.

WDSP TX chain (per CLAUDE.md §4.1 port table)

Source	Target (lyra-cpp)	Status	Notes
wdsp\TXA.c (channel scaffold)	src/wdsp/TxChannel.{h,cpp} + cffi cdef	■	TxChannel shipped on main (TX-1 components). wdsp_native.{h,cpp} carries 17 SetTXA*/GetTXAMeter cdefs. Reference-parity audit pass recommended for direct-port verdict.
wdsp\bandpass.c (bp0/bp1)	(consumed via WDSP cffi — SetTXABandpassFreqs)	■	The §15.23 trap (SetTXABandpassRun overrides bp1 to a stale state) is documented in CLAUDE.md + project_lyra_cpp_tx.md. cffi-only access is the correct posture.
wdsp\compress.c (TX compressor)	src/wdsp/Compress.{h,cpp} OR cffi binding	■	Operator choice: direct port (~150 LOC) for v0.2.1 EQ + dynamics ship, OR cffi-only binding if WDSP DLL exports are sufficient. Direct port recommended for parity with Lyra's existing UI control surface.
wdsp\cfcomp.c (5-band CFC)	src/wdsp/CFComp.{h,cpp} OR cffi binding	■	Same operator-choice. ~600 LOC; one of the heavier ports. Replaced in Python Lyra plan by a Lyra-native Combinator per §15.19; cpp may take the same path (custom Combinator instead of WDSP CFC) — operator design call.
wdsp\osctrl.c (CESSB)	cffi binding (SetTXAosctrlRun)	■	Likely cffi-only; ~200 LOC port not justified unless operator UI need surfaces.
wdsp\wcpagc.c mode 5 (leveler)	cffi binding (SetTXALeveler*)	■	cdef already present in wdsp_native.cpp. UI surface exists in TX-1 components on main. Confirm reference-parity at the call-site setter sequence.
wdsp\wcpagc.c mode 5 (ALC)	cffi binding (SetTXAALCMaxGain)	■	cdef present; SetTXAALCThresh does NOT exist (§15.23-class trap — only MaxGain governs ALC ceiling).
wdsp\eqp.c (parametric EQ)	src/wdsp/EQ.{h,cpp} OR cffi binding	■	Per CLAUDE.md §4.1 v0.2.1; ~300 LOC. Lyra-native parametric EQ likely (§15.19 lists 3-or-5-band parametric replacing the WDSP 10-band graphic).

Source	Target (lyra-cpp)	Status	Notes
wdsp\gen.c (gen0/gen1 generators)	ffi binding (SetTXAPreGen*/SetTXAPostGen*)	■	gen1 (postgen) used today for TUN tone. gen0 (pregen) cdefs deferred per §15.23 (bench-tooling nicety, not on the signal path).

TX PureSignal (per CLAUDE.md §4.1 v0.3)

Source	Target (lyra-cpp)	Status	Notes
wdsp\iqc.{c,h}	src/wdsp/Iqc.{h,cpp} (~315 LOC)	■	v0.3. ffi math + Lyra-cpp wrapper for the 5-state PS lifecycle (RUN, BEGIN, SWAP, END, DONE).
wdsp\calcc.{c,h}	src/wdsp/Calcc.{h,cpp} (~1164 LOC)	■	v0.3. ffi math + Lyra-cpp wrapper for the 8-state PS FSM + 3-state auto-attenuator FSM + PSDialog UI.
wdsp\lmath.c::xbuilder	src/wdsp/PsXBuilder.{h,cpp} (~200 LOC)	■	v0.3. Cubic-spline coefficient builder, used by Lyra-cpp-side calcc orchestration.
wdsp\delay.c	src/wdsp/DelayLine.{h,cpp} (~80 LOC)	■	v0.3. TX/feedback time-alignment.

Console (C# — study only, Lyra-native equivalents)

Source	Target (lyra-cpp)	Status	Notes
Console\console.cs::chkMOX_CheckedChanged2	src/ptt.{h,cpp} + src/wdsp_engine.cpp keydown/keyup	■	Lyra-native FSM shipped on main. Verified against the reference's keydown/keyup ordering invariants (RX-DSP stop on keydown → wire MOX → TX-DSP start; keyup TX-DSP off → mox_delay → clear MOX → ptt_out_delay → RX-DSP restart). No direct port — C# is study-only by license.
Console\console.cs (TR delays)	src/tx/TrSequencing.{h,cpp} (LIVE)	■	Lyra-native shipped with operator's Thetis-DB-verified defaults (mox=15/rf=50/space_mox=13/ptt_out=5/key_up=10 ms).
Console\console.cs::m_bATTonTX	src/tx/AttOnTxPolicy.{h,cpp} (LIVE)	■	Lyra-native shipped on main. Reference posture: keydown save per-band step-att + force-31 (when PS-A off OR CW); keyup restore. Implementation verified against console.cs:30293-30327 + :30391-30410.
Console\PSForm.cs	src/ui/PsDialog.{h,cpp} (NEW, v0.3)	■	v0.3 PureSignal UI. C# study only; write Qt/QML native.
Console\HPSDR\IoBoardHl2.cs	src/hl2_stream.cpp (LIVE)	■	HL2 I/O quirks already implemented Lyra-native via the protocol-byte verification + gateway-RTL ground-truth pass (CLAUDE.md §15.26).

■ STAGE INDEX — ordered work plan

Each stage is one focused sub-arc (1+ commits) with an operator HL2 bench gate. Earlier stages must complete before later stages start unless explicitly tagged "independent".

Stage	Status	What	Notes
A	■ DONE	CMaster.cpp wire-up stubs (Send*Outbound* + SetTCIRun)	Foundation that subsequent stages build on.
B	■ DONE	aamix.c direct port + wire into RX path	Shipped 2026-06-08, bench-validated end-to-end through both AK4951 and PC-Soundcard sinks. Commits 27ac2fa → 7a50b07 on tx-rebuild.
C	■ TBD	ilv.c direct port (TX I/Q interleaver)	Candidate next step. Pairs with the Stage-A SendpOutboundTx stub; required for the TX I/Q path to flow reference-faithfully into the EP2 packer. Ship pattern mirrors Stage B: header port w/ attribution → cpp port w/ idiom translations → wire-up at WdspEngine TX channel → operator HL2+ bench (TX I/Q on dummy load shows clean carrier at LO from a known mic input).
D	■ TBD	xcmaster pump body direct port	Replaces the current Lyra-native pump driver loop with the reference-verbatim version. Operator decision: do this OR keep the Lyra-native pump if reference-parity audit passes.
E	■ TBD	TxChannel reference-parity audit + reconciliation	TxChannel shipped on main (TX-1 components 2-6) was a forward-reasoning effort; this stage formally audits it against cmaster.c::create_xmtr lines 177-190 + wdsp/TXA.c::create_txa and merges any verified divergences.
F	■ TBD	WDSP TXA chain components per CLAUDE.md §4.1 v0.2.1	compress / cfcomp / eqp / wcpagc — operator-driven per-feature direct port vs cffi-only call. Order TBD.
G	■ TBD	PureSignal direct ports (v0.3)	iqc / calcc / xbuidler / delay. Lands when v0.3 PS work starts. Needs operator HL2+ with the PS hardware mod.
H	■ TBD	TX/PS auto-attenuator FSM port	C# study only (PSForm.cs); Lyra-native rewrite.

Stage ordering note: Stages C–H are not strictly linear. Operator picks which stage to land next based on what unlocks the most-useful operator-visible behavior. The methodology (smallest revertable step + bench gate) applies within each stage regardless of cross-stage ordering.

■ OPEN QUESTIONS FOR OPERATOR

These are decisions that shape the plan but I cannot make unilaterally — surface them when picking up tomorrow:

1. **Stage C (ilv.c) vs Stage D (xcmaster) vs Stage E

(TxChannel audit) — which first?** All three are defensible:

- Stage C unlocks reference-faithful TX I/Q flow.
- Stage D removes the Lyra-native pump (cleaner end-state).
- Stage E formalizes what's already shipped (lowest risk,

highest confidence-gain).

2. **Do we re-port networkproto1.c + network.c + the Step 14

wire pieces direct from reference, OR leave them as the current mix of Lyra-native rewrites + original `hl2_stream.cpp`?** The Step 14 plan was midway through these when Stage B started. Direct-port would replace the rewrites with reference-verbatim ports. Operator call.

3. **WDSP TXA-chain components — direct port (~150-600 LOC

each) vs cffi-only?** The DLL exports work; cffi is faster to ship. Direct port gives Lyra-cpp full source visibility + lets us patch behavior at the source level. Per-component operator decision; default = cffi-only unless a real source-level need surfaces.

4. **PS work timing — start v0.3 (iqc/calcc/etc.) now in

parallel with TX, or sequence after TX is fully done?** The CLAUDE.md plan defers PS to v0.3 (after v0.2 TX release). Operator may want to start the ports early.

5. **The §15.27 "opt-in toggles that become defaults" cleanup

from the Python project — does it have a lyra-cpp analog here?** The Python project had a long backlog of legacy toggles to retire. Lyra-cpp doesn't carry the same legacy, but new direct-port commits may obsolete some Settings surfaces operator-knows-about — track per-component.

■ SHIPPED-WORK LOG

Newest at top. Cross-references SESSION_LOG.md for full arc details.

2026-06-08 EOD — Stage B aamix.c port COMPLETE

• **Stage B (aamix.c)** ■ direct port shipped + bench-validated

• Commits 27ac2fa → 7a50b07 on tx-rebuild

• Operator HL2+ bench 19:27:49 → 19:28:47: audible RX through

both AK4951 jack and PC-Soundcard sinks; mute/unmute via dispatch; output-device swap; AGC mode changes; chain out=N tracks 1:1 with dispatch calls=N across both sinks

• B.6.b root cause: Stage-A SendpOutboundRx no-op stub at

`RadioNet.cpp:272` clobbered `aaMix_>Outbound` ~1 ms after `wdspEngine::openRx1` wired the real lambda. Fixed by deleting the stub (commit 8b8e0da).

• 2-stage diagnostic instrumentation (chain @ 225518c +

dispatch @ 12369bc) localized the defect to a single line of code on the 2nd bench — pure-speculation cost avoided.

• Methodology lessons locked into this doc § "LOCKED

METHODOLOGY" above (counters-first when symptom is "nothing happens"; operator-empirical rule held; "do as Thetis does" continues to pay).

- Tasks #130 (parent) + #132 (Stage B.6.b) closed.
- Backup _backups/lyra-cpp-2026-06-08-aamix-stage-B-COMPLETE-with-docs.bundle.

Earlier work

See SESSION_LOG.md for the full session-by-session log. The Step 14 wire-layer arc (Stages 1, 1.5, 2a, 2b1, 2b2) is covered in detail there + in EXECUTION_PLAN.md's progress- tracking sections. TX-1 components 2-6 (main branch) are covered in project_lyra_cpp_tx.md (memory file).

■ RESUME POINTER (next session)

First actions (locked discipline):

1. git fetch origin
2. git status + git log --oneline -5 tx-rebuild origin/main
3. Read this doc + the "READ FIRST" block in
project_lyra_cpp_tx.md
4. Read SESSION_LOG.md newest entry for arc detail

Then: wait for operator decision on which Stage (C / D / E / F / open-questions item) to land next. Each Stage gets its own grounded design + reference-read + smallest-revertable- step + bench-gated implementation pattern matching the Stage B arc.

Last updated: 2026-06-08 EOD — Stage B aamix.c port shipped + bench-validated. Next port target = operator's call.